



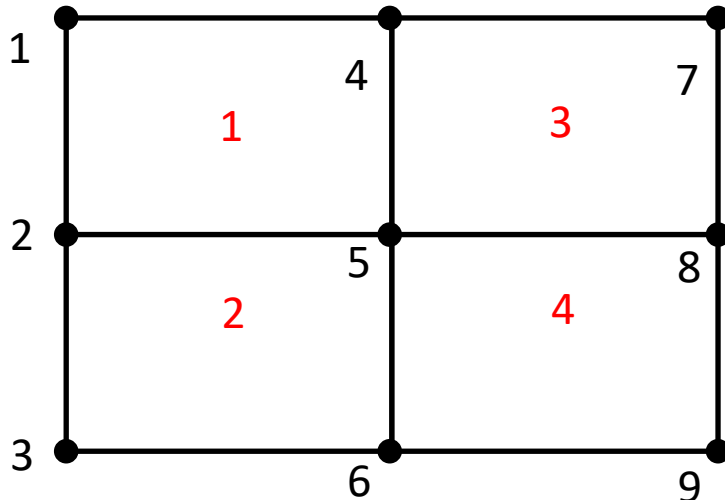
POLITECNICO
MILANO 1863

Topology optimization of 2D distributed systems

Introduction to the 99 lines code

The *99 lines* code is a working and complete (yet simple) topology optimization code written in Matlab.

The paper can be downloaded for free from [here](#).



- 2D planar structure;
- Design variables are densities of each element
- SIMP (solid isotropic material with penalization)
- Node and element numbering is column-wise from upper left corner

$$\text{SIMP: } \hat{E} = x^p E$$

x: element density (DV)

p: penalty exponent (typically 3)

E: material property (e.g. elastic modulus)

Lines 1 - 13

The code is a function: you must invoke it from a script or Command Window.

```
1  %%%% A 99 LINE TOPOLOGY OPTIMIZATION CODE BY OLE SIGMUND, JANUARY 2000 %%%  
2  %%%% CODE MODIFIED FOR INCREASED SPEED, September 2002, BY OLE SIGMUND %%%  
3  function x=top(nelx,nely,volfrac,penal,rmin)  
4  % INITIALIZE  
5  x(1:nely,1:nelx) = volfrac;  
6  loop = 0;  
7  change = 1.;  
8  % START ITERATION  
9  while change > 0.01  
10     loop = loop + 1;  
11     xold = x;  
12     % FE-ANALYSIS  
13     [U]=FE(nelx,nely,x,penal);
```

x: element densities

nelx: number of elements in the x direction
nely: number of elements in the y direction
volfrac: volume fraction
penal: penalty exponent
rmin: radius for the filter (discussed later)

Finite element analysis is run to obtain displacements of nodes (U)

Finite element subroutine, lines 66 - 100

```

66 %%%%%%%%%% FE-ANALYSIS %%%%%%%%%%
67 function [U]=FE(nelx,nely,x,penal)
68 [KE] = lk;
69 K = sparse(2*(nelx+1)*(nely+1), 2*(nelx+1)*(nely+1));
70 F = sparse(2*(nely+1)*(nelx+1),1); U = zeros(2*(nely+1)*(nelx+1),1);
71 for elx = 1:nelx
72     for ely = 1:nely
73         n1 = (nely+1)*(elx-1)+ely;
74         n2 = (nely+1)* elx +ely;
75         edof = [2*n1-1; 2*n1; 2*n2-1; 2*n2; 2*n2+1; 2*n2+2; 2*n1+1; 2*n1+2];
76         K(edof,edof) = K(edof,edof) + x(ely,elx)^penal*KE;
77     end
78 end
79 % DEFINE LOADS AND SUPPORTS (HALF MBB-BEAM)
80 F(2,1) = -1;
81 fixeddofs = union([1:2*2*(nely+1)],[2*(nelx+1)*(nely+1)]);
82 alldofs = [1:2*(nely+1)*(nelx+1)];
83 freedofs = setdiff(alldofs,fixeddofs);
84 % SOLVING
85 U(freedofs,:) = K(freedofs,freedofs) \ F(freedofs,:);
86 U(fixeddofs,:)= 0;
87 %%%%%%%%%% ELEMENT STIFFNESS MATRIX %%%%%%%%%%
88 function [KE]=lk
89 E = 1.;
90 nu = 0.3;
91 k=[ 1/2-nu/6   1/8+nu/8 -1/4-nu/12 -1/8+3*nu/8 ...
92     -1/4+nu/12 -1/8-nu/8  nu/6      1/8-3*nu/8];
93 KE = E/(1-nu^2)*[ k(1) k(2) k(3) k(4) k(5) k(6) k(7) k(8)
94                  k(2) k(1) k(8) k(7) k(6) k(5) k(4) k(3)
95                  k(3) k(8) k(1) k(6) k(7) k(4) k(5) k(2)
96                  k(4) k(7) k(6) k(1) k(8) k(3) k(2) k(5)
97                  k(5) k(6) k(7) k(8) k(1) k(2) k(3) k(4)
98                  k(6) k(5) k(4) k(3) k(2) k(1) k(8) k(7)
99                  k(7) k(4) k(5) k(2) k(3) k(8) k(1) k(6)
100                 k(8) k(3) k(2) k(5) k(4) k(7) k(6) k(1)];

```

x is the vector of densities

Line 68 invoke the function which builds the element stiffness matrix

Building up the stiffness matrix of the structure

Defining loads and boundary conditions: you might want to change this to study different structures

Elegant solution to the linear system $F = Ku$

KE is the stiffness matrix of the element. It is the same for all the elements, therefore it is computed just one time.

Finite element subroutine, lines 66 - 100

```

66 %%%%%%%%%% FE-ANALYSIS %%%%%%%%%%
67 function [U]=FE(nelx,nely,x,penal)
68 [KE] = lk;
69 K = sparse(2*(nelx+1)*(nely+1), 2*(nelx+1)*(nely+1));
70 F = sparse(2*(nely+1)*(nelx+1),1); U = zeros(2*(nely+1)*(nelx+1),1);
71 for elx = 1:nelx
72     for ely = 1:nely
73         n1 = (nely+1)*(elx-1)+ely;
74         n2 = (nely+1)* elx +ely;
75         edof = [2*n1-1; 2*n1; 2*n2-1; 2*n2; 2*n2+1; 2*n2+2; 2*n1+1; 2*n1+2];
76         K(edof,edof) = K(edof,edof) + x(ely,elx)^penal*KE;
77     end
78 end
79 % DEFINE LOADS AND SUPPORTS (HALF MBB-BEAM)
80 F(2,1) = -1;
81 fixeddofs = union([1:2*2*(nely+1)], [2*(nelx+1)*(nely+1)]);
82 alldofs = [1:2*(nely+1)*(nelx+1)];
83 freedofs = setdiff(alldofs,fixeddofs);
84 % SOLVING
85 U(freedofs,:) = K(freedofs,freedofs) \ F(freedofs,:);
86 U(fixeddofs,:)= 0;
87 %%%%%%%%%% ELEMENT STIFFNESS MATRIX %%%%%%%%%%
88 function [KE]=lk
89 E = 1.;
90 nu = 0.3;
91 k=[ 1/2-nu/6    1/8+nu/8 -1/4-nu/12 -1/8+3*nu/8 ...
92    -1/4+nu/12 -1/8-nu/8  nu/6      1/8-3*nu/8];
93 KE = E/(1-nu^2)*[ k(1) k(2) k(3) k(4) k(5) k(6) k(7) k(8)
94                  k(2) k(1) k(8) k(7) k(6) k(5) k(4) k(3)
95                  k(3) k(8) k(1) k(6) k(7) k(4) k(5) k(2)
96                  k(4) k(7) k(6) k(1) k(8) k(3) k(2) k(5)
97                  k(5) k(6) k(7) k(8) k(1) k(2) k(3) k(4)
98                  k(6) k(5) k(4) k(3) k(2) k(1) k(8) k(7)
99                  k(7) k(4) k(5) k(2) k(3) k(8) k(1) k(6)
100                 k(8) k(3) k(2) k(5) k(4) k(7) k(6) k(1)];

```

To further understand this code you might refer to the guide for commands:

sparse (line 69 and others)

setdiff (line 83)

mldivide (line 85) generally invoked using the shorter command \

sparse saves almost empty matrices in a memory-efficient way

setdiff compares two arrays and returns the different elements

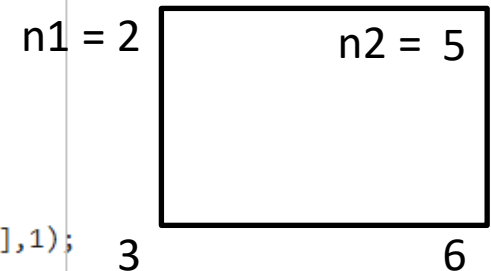
\ is used to solve linear systems $Ax = b$ such that:
 $x = A^{-1}b = A \backslash b$

Lines 14 - 25

Lines up to 13 have been commented already. After running FEA, the sensitivity analysis is performed.

```
8 % START ITERATION
9 while change > 0.01
10     loop = loop + 1;
11     xold = x;
12 % FE-ANALYSIS
13 [U]=FE(nelx,nely,x,penal);
14 % OBJECTIVE FUNCTION AND SENSITIVITY ANALYSIS
15 [KE] = lk;
16 c = 0.;
17 for ely = 1:nely
18     for elx = 1:nelx
19         n1 = (nely+1)*(elx-1)+ely;
20         n2 = (nely+1)* elx +ely;
21         Ue = U([2*n1-1;2*n1; 2*n2-1;2*n2; 2*n2+1;2*n2+2; 2*n1+1;2*n1+2],1);
22         c = c + x(ely,elx)^penal*Ue'*KE*Ue;
23         dc(ely,elx) = -penal*x(ely,elx)^(penal-1)*Ue'*KE*Ue;
24     end
25 end
```

n1 = upper left node of the element
n2 = upper right node of the element
Thanks to the easy numbering order the other two lower nodes are immediately identified (n1 + 1) and (n2 + 1).



Array of element sensitivities

$$\frac{\partial c}{\partial x_e} = -p(x_e)^{p-1} \mathbf{u}_e^T \mathbf{k}_0 \mathbf{u}_e .$$

In these lines sensitivities are filtered and the design variables are updated using the Optimality Criteria method. Then, results are printed.

$$\widehat{\frac{\partial c}{\partial x_e}} = \frac{1}{x_e \sum_{f=1}^N \hat{H}_f} \sum_{f=1}^N \hat{H}_f x_f \frac{\partial c}{\partial x_f}. \quad (5) \quad x_e^{\text{new}} =$$

The convolution operator (weight factor) $\hat{H}_f$ is written as

$$\hat{H}_f = r_{\min} - \text{dist}(e, f),$$

$$\{f \in N \mid \text{dist}(e, f) \leq r_{\min}\}, \quad e = 1, \dots, N,$$

$$(6) \quad \begin{cases} \max(x_{\min}, x_e - m) & \text{if } x_e B_e^\eta \leq \max(x_{\min}, x_e - m), \\ x_e B_e^\eta & \text{if } \max(x_{\min}, x_e - m) < x_e B_e^\eta < \min(1, x_e + m), \\ \min(1, x_e + m) & \text{if } \min(1, x_e + m) \leq x_e B_e^\eta, \end{cases} \quad (2)$$

```

26 % FILTERING OF SENSITIVITIES
27 [dc] = check(nelx,nely,rmin,x,dc);
28 % DESIGN UPDATE BY THE OPTIMALITY CRITERIA METHOD
29 [x] = OC(nelx,nely,x,volfrac,dc);
30 % PRINT RESULTS
31 change = max(max(abs(x-xold))); The variable change controls the end of the iterations
32 disp([' It.: ' sprintf('%4i',loop) ' Obj.: ' sprintf('%10.4f',c) ...
33       ' Vol.: ' sprintf('%6.3f',sum(sum(x))/(nelx*nely)) ...
34       ' ch.: ' sprintf('%6.3f',change) ])
35 % PLOT DENSITIES
36 colormap(gray); imagesc(-x); axis equal; axis tight; axis off; pause(1e-6);
37 end

```

End of the while loop

where m (move) is a positive move-limit, η ($= 1/2$) is a numerical damping coefficient and B_e is found from the optimality condition as

$$B_e = \frac{-\frac{\partial c}{\partial x_e}}{\lambda \frac{\partial V}{\partial x_e}}, \quad (3)$$

where λ is a Lagrangian multiplier that can be found by a bi-sectioning algorithm.

The filter alleviates the mesh sensitivity and helps to obtain a reasonable solution (avoids the formation of *chessboard* lattice).

```

50  %%%%%%%%% MESH-INDEPENDENCY FILTER %%%%%%%%%
51  function [dcn]=check(nelx,nely,rmin,x,dc)
52  dcn=zeros(nely,nelx);
53  for i = 1:nelx
54      for j = 1:nely
55          sum=0.0;
56          for k = max(i-floor(rmin),1):min(i+floor(rmin),nelx)
57              for l = max(j-floor(rmin),1):min(j+floor(rmin),nely)
58                  fac = rmin-sqrt((i-k)^2+(j-l)^2);
59                  sum = sum+max(0,fac);
60                  dcn(j,i) = dcn(j,i) + max(0,fac)*x(1,k)*dc(1,k);
61              end
62          end
63          dcn(j,i) = dcn(j,i)/(x(j,i)*sum);
64      end
65  end

```

← rmin is the filter size (divided by element size)
dc is the array of element sensitivities

$$\widehat{\frac{\partial c}{\partial x_e}} = \frac{1}{x_e \sum_{f=1}^N \hat{H}_f} \sum_{f=1}^N \hat{H}_f x_f \frac{\partial c}{\partial x_f}. \quad (5)$$

The convolution operator (weight factor) $\hat{H}_f$ is written as

$$\hat{H}_f = r_{\min} - \text{dist}(e, f),$$

$$\{f \in N \mid \text{dist}(e, f) \leq r_{\min}\}, \quad e = 1, \dots, N, \quad (6)$$

Note: floor(1.9) = 2

This function solves the OC using the bisection method (numerical method).

```

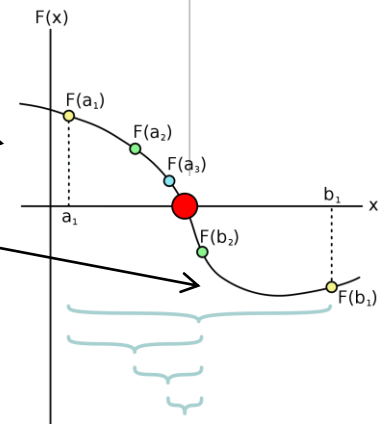
38 %%%%%%%%%% OPTIMALITY CRITERIA UPDATE %%%%%%%%%%
39 function [xnew]=OC(nelx,nely,x,volfrac,dc)
40 l1 = 0; l2 = 100000; move = 0.2;
41 while (l2-l1 > 1e-4)
42     lmid = 0.5*(l2+l1);
43     xnew = max(0.001,max(x-move,min(1.,min(x+move,x.*sqrt(-dc./lmid)))));
44     if sum(sum(xnew)) - volfrac*nelx*nely > 0
45         l1 = lmid; — Solution is at the right of the average value
46     else
47         l2 = lmid; — Solution is at the left of the average value
48     end
49 end

```

$x_e^{\text{new}} =$

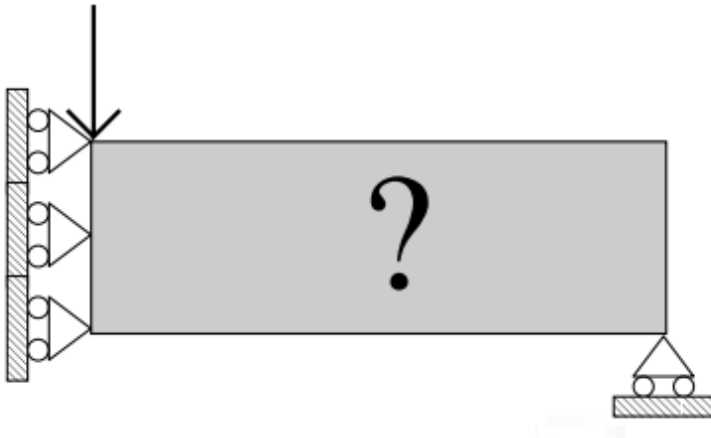
$$\begin{cases} \max(x_{\min}, x_e - m) & \text{if } x_e B_e^\eta \leq \max(x_{\min}, x_e - m), \\ x_e B_e^\eta & \text{if } \max(x_{\min}, x_e - m) < x_e B_e^\eta < \min(1, x_e + m), \\ \min(1, x_e + m) & \text{if } \min(1, x_e + m) \leq x_e B_e^\eta, \end{cases}$$

(2)



First request

Run the 99 line function. It solves this structure.
Can you imagine the optimized shape?



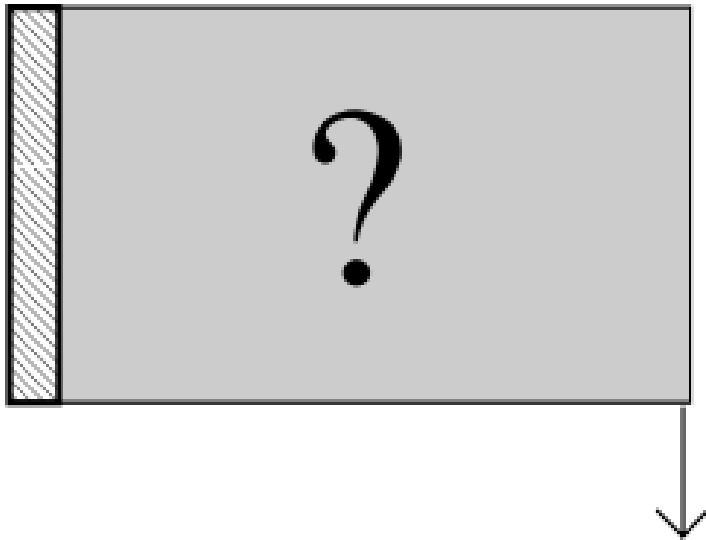
Use the following data:
`top(nelx,nely,volfrac,penal,rmin) =`
`top(60,20,0.5,3.0,1.5)`

Try different values of **rmin** and **penalty**
to see the difference in the results.
Hint: Modify one parameter per time.

Second request

Run the 99 line function in order to solve this structure.

You have to modify the code to account for boundary and loading conditions. (Hint: lines 79 and 80)



```
79 F(2*(nelx+1)*(nely+1),1) = -1;  
80 fixeddofs    = [1:2*(nely+1)];
```

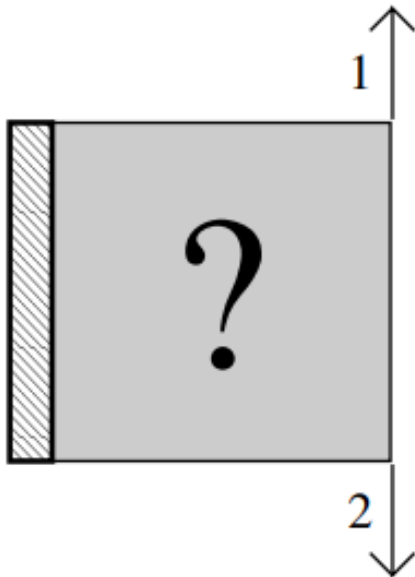
Use the following data:

```
top(nelx,nely,volfrac,penal,rmin) =  
top(32,20,0.4,3.0,1.2)
```

Third request

Run the 99 line function in order to solve this structure.

In this case, you'll see the difference in optimized structure when applying the loadcases simultaneously or one per time. Study this two scenarios:



- A single load case (forces are applied simultaneously)
- Two load cases (forces are applied one per time), like in the training lab of the motorcycle frame.

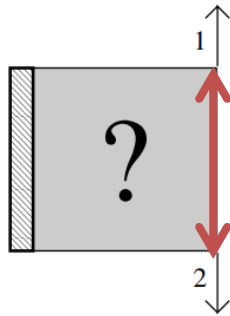
Hint: **you have to update the way compliance is computed**

Use the following input:
`top(30,30,0.4,3.0,1.2)`

Third request

Scenario 1

Forces are applied simultaneously:

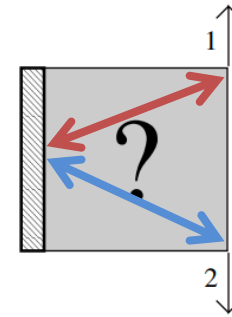


Path of the force

Reaction forces for this scenario are null

Scenario 2

Forces are applied with two load cases:



Paths of the forces

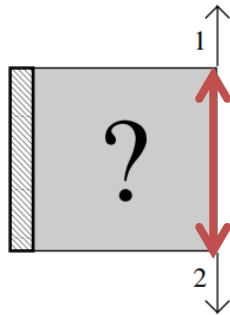
Reaction forces for this scenario are not null

i.e. the **topology** of the problem is different!!!

Third request

Scenario 1

Forces are applied simultaneously:



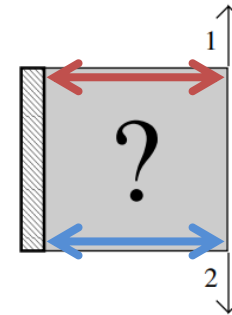
$$C = (\mathbf{F}_1^T + \mathbf{F}_2^T) \cdot \mathbf{u}$$

Solve this case modifying the vector of forces

```
79 F(2*(nelx+1)*(nely+1),1) = -1.;  
    F(2*(nelx)*(nely+1)+2,2) = 1.;
```

Scenario 2

Forces are applied with two loadcases:

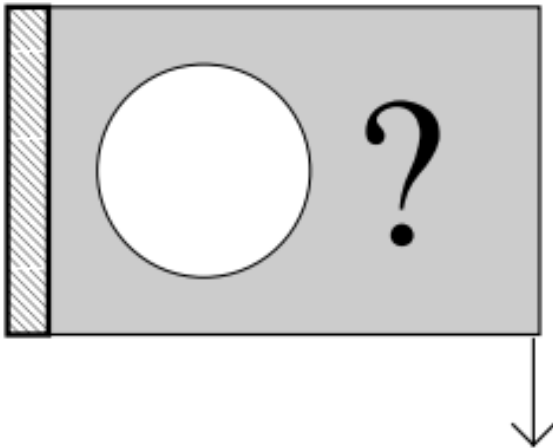


$$C = w_1 \mathbf{F}_1^T \cdot \mathbf{u}_1 + w_2 \mathbf{F}_2^T \cdot \mathbf{u}_2$$

Solve this case modifying the vector of forces **and** the way compliance is computed (use weights equal to one).

Fourth request

Run the 99 line function in order to solve this structure. This structure presents a **hole** which must be **preserved** during the iterations.



Hint: after updating the design variables array, set the density of elements forming the hole to a fixed and very low value (which represents void).

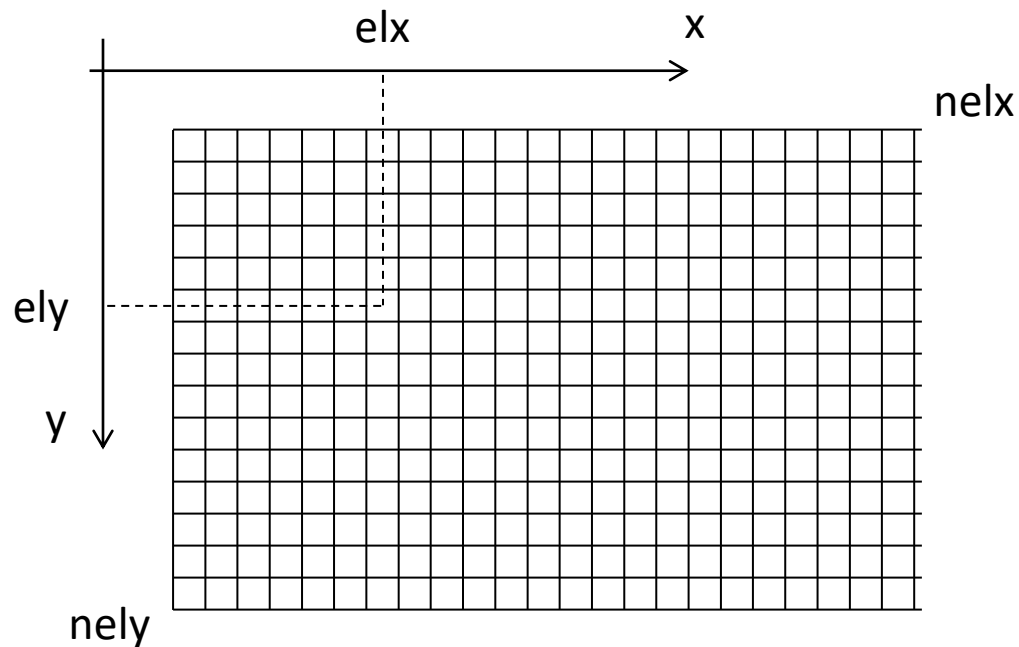
Use the following input:
`top(45,30,0.5,3.0,1.5)`

Fourth request

To preserve geometry, i.e. the hole remains a hole over iterations, you should proceed this way:

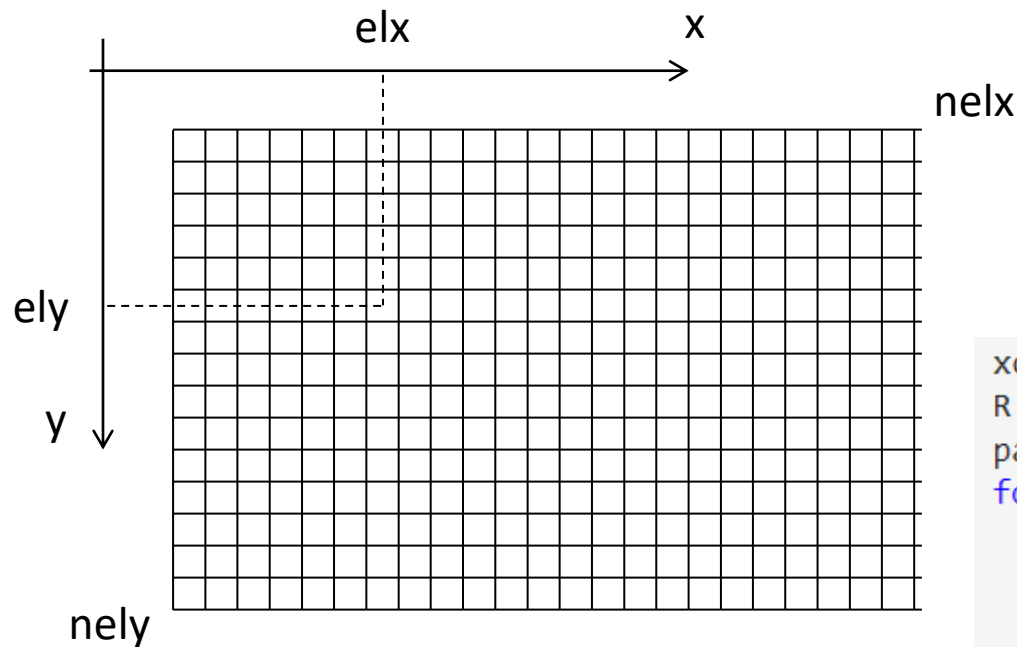
1. Generate an array that defines which elements must be preserved (in this request *preserving an element* means *preserving its void*). This array should act as a map that restores the properties to be preserved for the required elements.
2. Use the preserving array to correct, after each iteration, the value of the density. (Density must be very low, to simulate void).

Fourth request



The elements (and nodes) sorting method automatically defines a metric for distances and lengths. Each element has a position defined by its index (elx , ely). In the example on the left, the highlighted element has a position (7, 6).

Fourth request



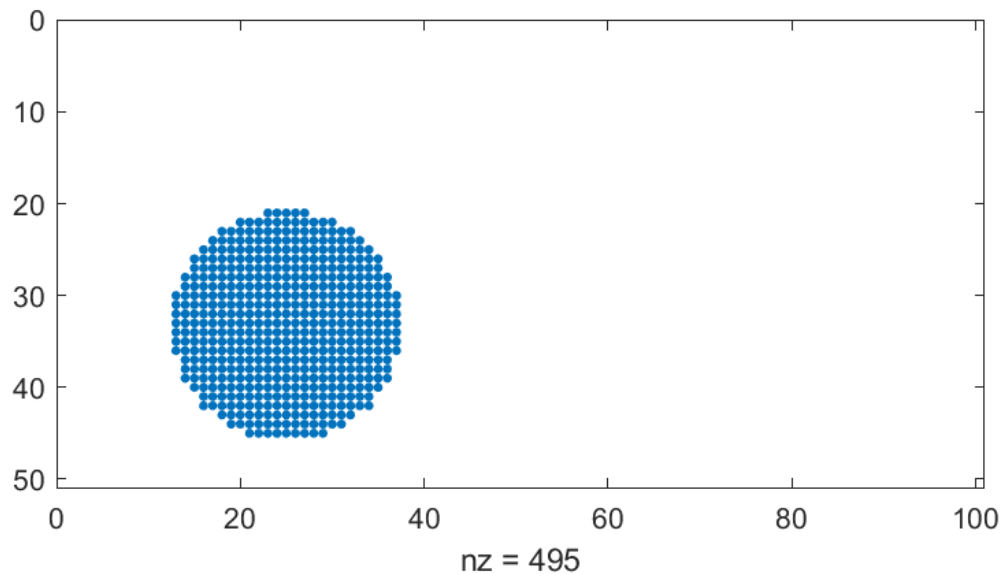
Once you have decide a radius and a location for the center of the hole, you create a *passive* array which preserves all the elements whose distance with respect to the center of the hole is less than the radius.

```
xc = nely/2; yc = nelx/3;  
R = nely/4;  
passive = zeros(nely, nelx);  
for elx = 1 : nelx  
    for ely = 1 : nely  
        if (elx - xc)^2 + (ely - yc)^2 > R^2  
            continue  
        end  
        passive(ely, elx) = 1;  
    end  
end
```

Fourth request

After each iteration, you should use the *passive* array to correct the values of densities for elements which must be preserved:

```
xnew(passive) = 0.001;
```



Example of passive array;
`spy(passive)` is shown.
Blue dots correspond to elements
to be preserved (values 1).

Fourth request

Logical flowchart

